

고정점을 이용한 프로그래밍 언어 거리 측정(Measuring the Distance of Programming Languages Using Fixed-Points)

한상곤, 천준석, 우균 (부산대학교 대학원 정보융합공학과)

연구 배경

- 프로그래밍 언어 사이의 유사도는 암묵적인 기준이 대부분이며, 정량적 기준이 부재함
- 대규모 언어 모델(LLM)을 활용한 언어 간 거리의 정량적 측정법 제안

핵심 가설

- 자연어 처리의 평가 기법인 왕복 번역(Round-trip Translation, RTT)을 코드에 적용하여 프로그래밍 언어 간 거리를 정량적으로 도출할 수 있음¹

연구 목적

1. 프레임워크 가능성 탐색: C++를 기준으로 C, Java, Python을 오가는 왕복 번역 측정
2. LLM 모델 간 특성 비교: 오픈소스 모델(qwen2.5-coder)과 상용 모델(gpt-5.4)의 특성 및 번역 양상 대조

1: 조찬우, 김영훈, 우균, "왕복 번역과 고정점을 이용한 언어 거리 계산," 2025 한국정보과학회 학술발표논문집, 2025.

왕복 과정에서 발생하는 정보 손실의 크기가 곧 두 언어 사이의 "거리"를 반영한다는 가설

- 자연어 처리(NLP)에서 번역 품질을 평가하는 비지도(unsupervised) 기법(한국어 → 영어 → 한국어로 번역 후 원문과 비교)
- ICML'24에서 Allamanis 등이 제안한 Round-trip Correctness(RTC)는 벤치마크 없이 코드 LLM을 평가할 수 있는 방법론(예측(코드 → 설명) → 역변환(설명 → 코드) → 테스트를 통한 의미 동치 검증)
- 코드 RTT 과정에서 필연적으로 발생하는 정보 손실(Information Loss) 과 구조적 변형을 두 언어가 공유하는 근본적 교집합의 크기로 보고 이를 언어 간 거리로 치환함²
- 의미 보존 정도를 측정하여 두 프로그래밍 언어 간의 구조적 이질성을 정량화함

2: Miltiadis Allamanis, Sheena Panthaplackel, Pengcheng Yin, "Unsupervised evaluation of code LLMs with round-trip correctness," ICML'24, pp. 1050–1066, 2024.

관련연구: 고정점(Fixed-point) 이론 기초

함수 f 에 대해 $f(x) = x$ 를 만족하는 값 x 를 고정점이라 함

- 코드를 반복적으로 왕복 번역($A \rightarrow B \rightarrow A \rightarrow B \rightarrow \dots$)할 때 더 이상 코드가 변하지 않는 상태
 - 0회 (원본): `int a = 0; for(...)`
 - 1회 왕복: `int a = 0; while(...)` (구조 변형 발생)
 - 2회 왕복: `int a = 0; while(...)` (변화 없음 $\rightarrow$ 고정점 도달)
- 고정점까지의 반복 횟수가 적을수록 두 언어가 구조적으로 유사함을 시사

코드를 반복 변환 시 더 이상 코드가 변경되지 않는 상태인 고정점에 도달하는지 여부 확인

- 고정점에 도달하기까지 걸리는 반복 횟수(Iteration count)
- 원본 코드와 수렴된 코드 간의 형태적 차이

실험 대상 코퍼스

- 백준 온라인 저지(BOJ)의 5개 알고리즘 문제
 - 문자열 탐색(1435), 이분 탐색(2110), 그리디(2217), 동적 프로그래밍(2579), 그래프 탐색(5567)

평가 기준 언어 및 구조적 이질성

- 기준 언어: C++
- 대상 언어: C (C++의 부분 집합), Java (객체지향), Python (동적 타입 및 다른 구문 체계)

평가 지표

1. 변경 횟수 거리: 고정점 도달까지 완료된 왕복 주기 수
2. MOSS 유사도³: 스탠포드 MOSS 시스템을 사용해 원본 C++와 왕복 후 C++ 코드 비교
3. 의미 보존 여부: 왕복 코드가 테스트 입출력을 모두 통과했는지 여부

3: A. Aiken, "Moss: A System for Detecting Software Plagiarism," 2004.

실험 결과: 모델별 컴파일/의미 보존 통과율 및 변경 횟수

언어 쌍	qwen2.5 평균 반복 횟수 및 통과율	gpt-5.4 평균 반복 횟수 및 통과율
C++ → C	2.2회 (100%)	5.4회 (100%)
C++ → Java	2.4회 (100%)	1.2회* (0%)
C++ → Python	2.2회 (100%)	1.0회* (0%)

- **qwen2.5** : 언어 종속성 없이 균일한 수렴 속도(2.2~2.4회)
- **gpt-5.4** : C 구문에서는 100% 보존 성공했으나 높은 반복 횟수(5.4회) 필요, Java 및 Python 같이 구조적 이질성이 큰 구문에선 실패(0%)

*: 대부분 1회차 실패로 인한 보존 불가

실험 결과: MOSS 유사도 분포

언어 쌍	모델	MOSS 유사도 (평균 ± 표준편차)
C++ → C	qwen2.5	19.3 ± 22.5%
	gpt-5.4	25.4 ± 24.4%
C++ → Java	qwen2.5	63.8 ± 14.2%
	gpt-5.4	36.7 ± 41.0%
C++ → Python	qwen2.5	0.0 ± 0.0% (전면 재작성)
	gpt-5.4	36.4 ± 23.8%

- **qwen2.5** : Python 변환 시 전면 재작성 수준의 구조 변화가 일어났으나 전반적으로 의미 보존 및 작동이 안정적임
- **gpt-5.4** : 원본 코드의 뼈대와 문법을 최대한 유지하려는 경향성

1. 기존 방법은 정적 특성(문법, 키워드)에 의존하였지만, 본 연구는 LLM의 동적 번역 과정에서 나타나는 정보 손실을 거리로 사용하여 런타임 의미 보존까지 검증하는 실행 기반(execution-based) 평가를 통해 프로그램 변환 과정의 "언어 사이 거리 정량 측정법"을 제시하였다.
2. 실험 결과, `qwen2.5` 은 범용적 번역 능력으로 안정적인 성능을 보였으며, `gpt-5.4` 는 언어 특성에 따른 큰 성능 편차를 보였다.
3. 향후 계획은 대규모 문제 셋을 활용한 실증 검증과 언어 구조 유지 vs 단순 실행 가능성 보장 등 요구사항에 따른 최적 모델 자동 할당 프레임워크 제안 등을 연구할 예정

Q&A

부록 A: MOSS(Measure of Software Similarity)

- 스탠포드 대학교 Alex Aiken 교수가 개발한 소프트웨어 유사도 측정 시스템 → 프로그래밍 과제의 표절 탐지

동작 원리

1. 전처리: 공백/주석 제거, 변수명을 일반 토큰으로 치환 (변수명 변경에 강건)
2. K-gram 생성: 토큰화된 코드를 연속 k 개 토큰 단위로 분할
3. Winnowing 알고리즘: 해시 윈도우를 슬라이딩하며 대표 해시(fingerprint)를 선택
4. 비교: 두 코드의 fingerprint 집합 간 공통 요소 비율로 유사도 산출

부록 B: 선정한 5개 문제의 알고리즘 유형

문제 번호	알고리즘	특성
1435	문자열 탐색	언어별 문자열 처리 방식 차이 관찰
2110	이분 탐색	수치 연산 중심의 알고리즘
2217	그리디	정렬·탐욕적 선택 패턴
2579	동적 프로그래밍(DP)	재귀/메모이제이션 구조 차이
5567	그래프 탐색	자료구조(인접 리스트 등) 표현 차이

부록 C: 실험에 사용된 LLM 모델 비교

항목	qwen2.5-coder	gpt-5.4
개발사	Alibaba Cloud	OpenAI
유형	오픈소스	상용(API)
모델 크기	0.5B ~ 32B	비공개
지원 언어	40개+ 프로그래밍 언어	다수
주요 강점	코드 생성, 수정, 추론에 특화	범용 추론, 강력한 언어 이해력

부록 D: 프로그래밍 언어 간 거리 측정

접근법	설명
키워드 집합 비교	예약어 집합 간 Jaccard 유사도 측정 (단순하지만 의미적 깊이 부족)
AST 구조 비교	추상 구문 트리의 편집 거리(TSED)로 구조적 차이 측정
벡터 임베딩	코드 스니펫을 벡터 공간에 매핑, 코사인 유사도로 비교
특성 벡터	타입 시스템, 패러다임, 메모리 관리 등 특성을 수치화하여 유클리드 거리 계산